Guía del notebook `1_08_adaline`: regresión con una neurona

Puntos: demostración, sin `TODO` · **Tiempo:** 10–15 min · **Clase:** 4 · **Archivo:** `notebooks/1_08_adaline.ipynb`
Teoría completa: Guía de la Clase 4, sección B4.

¿Qué vas a aprender?

- Que una neurona con **activación lineal** (ADALINE) sirve para **predecir números** (regresión).
- A ver el error como una **superficie** (un valle en 3D) que el descenso por gradiente tiene que bajar.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_08_adaline.ipynb` → kernel **nni**. El notebook lee `files/Salary_dataset.csv`, así que hay que abrirlo desde la carpeta del repo.
3. Este notebook **no tiene** `TODO`: ejecútalo completo y lee cada resultado.

La idea en 2 minutos

- **ADALINE** (Widrow y Hoff, 1960): $\hat{y} = w \cdot x + b$, sin función escalón ni sigmoide. Es una **recta**.
- **Error:** el error cuadrático medio (MSE) entre los salarios reales y los predichos.
- **Regla** (descenso por gradiente con $f' = 1$): $w \leftarrow w + \eta \cdot (y - \hat{y}) \cdot x$ y $b \leftarrow b + \eta \cdot (y - \hat{y})$.

Recorrido por las celdas

1. **Datos:** 30 personas, con sus años de experiencia y su salario. La gráfica muestra que a más experiencia, más salario.
2. **Clase `ADALINE`:** pesos al azar entre 0 y 1; `predict` calcula $w \cdot x + b$.
3. **Recta sin entrenar:** casi plana y pegada a 0, muy lejos de los puntos (los salarios valen decenas de miles).
4. **Superficie del error:** para muchas combinaciones de peso y sesgo calcula el MSE y lo dibuja en 3D. El punto más bajo de ese valle son los mejores parámetros.
5. **Entrenamiento:** 1000 pasadas con tasa 0.0001, ejemplo por ejemplo.
6. **Historial del MSE:** baja rápido y luego se aplana.
7. **Recta final:** pasa por en medio de los puntos.

✅ **Qué esperar:**

- El resultado cambia un poco cada vez, porque los pesos iniciales son al azar (no hay semilla). En una ejecución salió $w \approx 8\,400$ y $b \approx 32\,000$.
- La recta **óptima** (mínimos cuadrados) es **salario ≈ 9 450 · años + 24 848**. `docs/practica_04.py` llega exactamente a ella.

Para pensar

¿Por qué el notebook no llega exactamente a la recta óptima?

- Los datos no están **escalados**: los años van de 1 a 10.6 y los salarios de 37 000 a 122 000.
- Por eso hay que usar una tasa diminuta (0.0001), y con ella el **sesgo** aprende muy lento.
- Si se escalan los datos (restar la media y dividir entre la desviación estándar), el descenso llega al óptimo rápido. Eso hace `docs/practica_04.py`.

Pruébalo:

```
python docs/practica_04.py --graficar
```

Errores frecuentes

Síntoma	Causa	Solución
`FileNotFoundError: files/Salary_dataset.csv`	Notebook abierto fuera de la carpeta	Abre la carpeta `curso-redes-neuronales`
La gráfica 3D no aparece	Falta matplotlib, o la celda dio error	`pip install matplotlib` con `(nni)` activo
Recta distinta a la de un compañero	Pesos iniciales al azar	Es normal; agrega `np.random.seed(0)` al principio para repetir resultados